

The road to ALAKAZAM — how the models actually connect

Written 2026-07-28. Every number here comes from a run.

The one-sentence diagnosis

We have four models and a playing bot that uses one of them.

MAG plays. PORYGON2, SLOWKING and DODUO are fitted artifacts sitting on disk that have never been inside a live decision. That is not a small gap — it is the same failure that hid DODUO for a week: fitted, saved, measured on paper, never once in the loop.

What ALAKAZAM actually is

Four jobs. Poker has the same four.

job	who does it	poker
narrow the options worth thinking about	MAG	you don't solve all 1,326 hands
guess what they'll do	MAG-as-sampler	range vs range
evaluate the position you land in	PORYGON2	how good is this river
solve the simultaneous guess	SLOWKING	the actual solver

ALAKAZAM is those four running together on a live turn. Right now **only the first is connected**. MAG scores every legal move on 48 features, takes a weighted roll, clicks. No lookahead. It never asks "and then what."

Your four questions, answered

1. "Are all 48 features fully tested?"

Two different questions.

Test coverage: no. 8 of 48 features are named anywhere in `tests/`. 40 are not.

Reachability: all 48 pass, measured over 8,300 real decisions. Zero DEAD (same value on every option of every decision). Zero NO CONTRAST. Two RARE (`deadWeather` fires on 0.6% of decisions, `movesLowerFoe` 0.8%).

The NO-CONTRAST check is the one worth understanding. MAG scores options *against each other* and takes a softmax, which is mathematically invariant to anything added equally to every option. A feature that varies across the whole corpus but is constant *within* each decision looks perfectly healthy in a weight table and **cannot change a single click**. Zero features fail that. Genuinely good, and not guaranteed — this repo has shipped a mechanics document that was a regex artefact twice, switch features that were fitted and unreachable, and a joint layer that silently fell back on every eligible turn.

Stated plainly: this is not correctness. A feature can be alive, discriminating, frequent, and measuring the wrong thing. That needs a hand-built scenario per feature. The audit quantifies the test debt; it does not pay it.

2. "Does it need to switch? That seems deeply flawed."

You're right, and it's worse than the old justification admits.

The record says switching cost 10 points, so it was turned off — **RETRACTED 2026-08-06 as unattributable and confounded** (medicham2 payouts predating WIRES 123-128, no `engine_release` stamp, and `bringIn()` picks `live(bench)[0]` so it measured switching to an arbitrary body, which the engine itself distinguishes from a switch; see #63). But look at *which* weights that was measured on — the imitation fit, where the switch features came out as `switchSurvives1 +0.055`, `switchFaster -0.112`, and `switchSurvives2` with an interval containing zero. **Weights fitted out of noise, then acted on 4.4 times a game.** "Switching is bad" was really "switching *on those numbers* is bad."

And here is the part I had not checked until you asked: **MACHAMP has never trained switching.** `engine/ladder.js` spawns `mew.js` without `--switching`, so switch candidates were never built during the entire search. Those four weights were never under selection — they drifted on random perturbation and nothing selected against them. Comparing them anyway:

	shipped	MACHAMP
<code>isSwitch</code>	-0.063	-0.123
<code>switchSurvives1</code>	+0.183	+0.257
<code>switchSurvives2</code>	-0.034	-0.216
<code>switchFaster</code>	-0.148	+0.316

`switchFaster` flipped sign. That difference is **noise, not evidence** — it has to be, because nothing was selecting on it. The honest statement is: **we have never once optimised switching for winning.** A bot that cannot pivot has a hole a human finds in one game.

3. "What happened to searching for a similar scenario and choosing the highest-EV outcome?"

That is **PORYGON2**, and it exists — a k-NN value function that finds positions like this one and reads off how they ended. Trained on self-play, validated on held-out human games: 63.6% against a 60.5% material baseline.

It has never been connected to the playing bot. MAG has no `require` of it. The only value function anywhere in `magnemite.js` is `variance.js` 's win probability, and that is used solely by the risk lever, which is off by default.

So the thing you're describing — look ahead, find similar spots, take the best expected outcome — **is not what the bot does.** It scores this turn's moves on features and clicks. That gap is the single biggest architectural hole in the project, bigger than any feature.

There is also a reason PORY's learning curve is flat (eight times the data moved it *backwards* slightly): it learns from self-play games generated by an imitation policy. More games of two mediocre bots imitating humans does not teach you what a good position looks like. **PORY is not data-starved, it is policy-starved.**

4. "We need to update models like SLOWKING with the new framing."

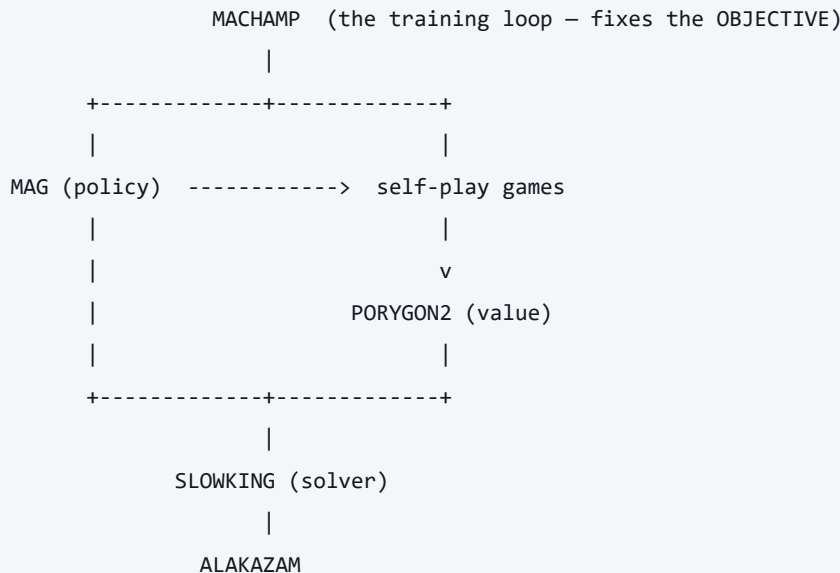
Agreed, and two things there are stale:

- The MODELS.md entry still claims a **non-transitive cycle** in the metagame. That was built on the old playstyle classifier — the one with five illegal species and a both-teams bug. The claim is disproven and the entry has not been corrected.
- SLOWKING's **confidence intervals are known invalid.** The bootstrap solves at 1,000 iterations while the point estimate solves at 15,000, so the interval measures solver convergence error rather than

uncertainty. Diagnosed, unfixed, your call.

The deeper reframe: SLOWKING was specified as the endgame solver, but a solver is only as good as the two things it consumes. Feed it an imitation policy and an imitation value function and it will solve the wrong game, precisely and expensively.

The dependency chain



Read it top-down: **MACHAMP improves MAG → better MAG generates better games → better games lift PORY → SLOWKING becomes worth running → ALAKAZAM is the three together.**

Read it bottom-up and you get the trap the project is currently in: you cannot fix PORY by feeding it more data, and you cannot fix SLOWKING by making the solver smarter.

Phases, each with a gate it must clear

Phase 0 — where we are

MAG plays alone. No lookahead, no value function, no coordination, no switching. MACHAMP's champion beats the imitation MAG 57.3%. WOBBUFFET is finding challengers that beat MAG above 60%, so it is still readable.

Phase 1 — give the bot a future (*unblocks your actual question*)

Wire PORYGON2 in as a leaf evaluator so the bot computes an expected value instead of a score. The opponent-sampler work is already done: MAG's move distribution is calibrated well enough to sample from, which was the blocker.

$EV(a) = \text{sum over their likely replies of } P(\text{reply}) \times V(\text{position after})$

Gate: beats the no-lookahead bot head-to-head, paired. If it does not, say so and stop — the same discipline that killed DODUO.

Phase 2 — let it switch, under the right objective

Re-run MACHAMP **with** `--switching on`, so the four switch weights are actually under selection for the first time.

Gate: the switching champion beats the non-switching champion. This is the first honest test of whether the 10-point penalty was about switching or about bad weights.

Phase 3 — fix what PORY eats

Regenerate PORY's training games from the MACHAMP champion rather than the imitation vector, then refit.

Gate: the learning curve stops being flat. If more data still does nothing, the problem is the state representation, not the policy — and that is a different fix.

Phase 4 — SLOWKING does the turn, not just the preview

Preview-Nash already works on real data. The in-battle solver does not. Wire `ChampionsGame` to the real engine, use MAG to narrow the branches and PORY at the leaves, and solve the simultaneous move properly instead of taking an argmax.

Gate: lower exploitability than the champion under WOBBUFFET. Not a higher win rate — this is the piece whose whole purpose is being hard to counter.

Also in this phase: correct the disproven cycle claim, and fix the invalid intervals.

Phase 5 — ALAKAZAM

Continual re-solve on a live turn: belief state over what they kept, re-solve each turn, output the move and the honest win percentage.

Gate: beats you. Not a bot — you.

What I would do next, and why

Phase 1. It is the thing you just asked about, it is the biggest architectural hole, and every one of its inputs is already built and measured. The bot has no concept of "and then what," and nothing else on the list matters as much.

Phase 2 is a close second because it is cheap — it is one flag on a run we already know how to do, and it settles a question that has been answered wrongly for a week.

The rule that governs all of it

Every phase above has a **gate**, and the gate is a measurement against a control, not a plausibility argument. DODUO had the better argument and lost. The mechanics batch had the better argument and did nothing. The forfeit hypothesis was mine and was wrong.

Nothing gets wired in permanently because it should work.